Instructor Handout 2: Operating-System Operations and System Protection

Auqib Hamid Lone
Department of Computer Science & Engineering, GCET Ganderbal

October 17, 2026

Instructor Copy — Not for Student Distribution

Concepts That Click

2.1 Before You Teach This Lecture

- **Silberschatz, Galvin & Gagne, *Operating System Concepts*, 8e** — Ch. 1 §1.5, pp. 20–23: dual-mode operation, the timer, and the protection mechanisms built on them. This is the primary anchor for the whole lecture. *Page-verified.*
- **Stallings, *Operating Systems: Internals and Design Principles*, 6e** — Ch. 3 §3.5, p. 140: execution of the operating system (when and how the kernel runs, and the mode switches that get it there). Ch. 3 §3.6, p. 143: security issues, the source for the protection-versus-security distinction. *Page-verified.*
- **Prepare the live demo and name one concrete machine** (Linux, Windows, macOS): have a terminal ready to launch a runaway process and kill it from a second terminal, and refer to that OS whenever you say "the kernel." Rehearse so the demo costs ninety seconds. Optional chapter-level colour for the trap path: Bovet & Cesati, *Understanding the Linux Kernel*, 3e, Ch. 4 and Ch. 10 (not page-cited here).
- **Prerequisite check and arithmetic.** Students met kernel-versus-shell in Lecture 1 and saw registers, interrupts, and I/O in CS401; recall those in two minutes. Do the 12 μs/Q derivation cold before class — the 1 μs case (1200%) must land without hesitation. This lecture assumes the system-call *idea* but defers the mechanics to Lecture 3.

2.2 The Control Problem

Open with the story, not the definition. A student logs in to a shared laboratory machine and runs `while(1);`. The condition is the constant 1, the body is empty, so the processor executes the test and the branch forever. The program performs no input, produces no output, issues no system call, and never exits. On a single-core machine two programs cannot both be "running" at one instant: the CPU executes one instruction stream at a time. The prediction seems unavoidable — the machine should freeze. It does not. The instructor, from another terminal, can still list the process and kill it. This lecture explains both facts.

The explanation is not that the OS watches from outside. The OS is a program that must run on the same CPU as the code it supervises, using the same registers and physical memory.

It cannot run "in the background" while user code runs. The kernel therefore runs only when an **event** transfers control to it: a system call, an interrupt, or an exception. Between events it is not executing at all. That fact produces the central design problem, which is worth writing on the board verbatim:

> *Build a machine that can run untrusted code at full speed, yet guarantee that the trusted kernel can always regain control.*

Every mechanism in the lecture is an answer to that sentence. Keep the sentence visible for the whole class; refer back to it when the timer arrives.

> **Teaching note:** Resist defining "mode bit" in the first five minutes. Let the class sit with the paradox and propose resolutions; the hardware answer then resolves a puzzle they have already felt, which is more durable than a definition delivered cold.

What goes wrong without a boundary

Without a boundary, four failures follow immediately. A buggy pointer writes over another process's memory — or over the kernel itself — because nothing checks the address. A malicious program disables interrupts, and the timer can no longer fire. A program reprograms the timer never to expire, then loops forever. A program issues raw disk commands, bypassing the file system and every access-control check. The lesson is blunt: no boundary means no isolation, no multiplexing, and no trust.

Board: Draw the two-instruction loop, not the whole program. On the board, in order: (1) write `L1:` at the left margin; (2) write `cmp 1, 0` and `jmp L1`; (3) circle both instructions and label the circle "user mode, mode bit = 1"; (4) write "no trap, no I/O, no privileged instruction" underneath. Narration: "The test is arithmetic, the branch is a branch — both unprivileged. Every iteration is legal, so the hardware has no reason to stop it."

Board: Build the failure ledger as a two-column table. On the board, in order: (1) draw the headers "Failure" and "Mechanism that prevents it"; (2) fill only the left column — buggy pointer, disable interrupts, reprogram the timer, raw disk command; (3) ask the class for the right column before writing it; (4) record the answers: memory protection, privileged instruction, privileged timer load, privileged I/O. Narration: "Each row is a different resource, but every fix moves the dangerous operation behind the boundary." Protection is a family of mechanisms, not one switch.

> **Teaching note:** Ask the predictive question before the ledger's right column exists: "Of these four failures, which one can dual-mode operation alone stop?" Students usually say "all of them." The correct answer is the first three kinds of misuse of *authority*; the runaway loop in the next section is the case dual-mode cannot stop, because the loop never asks for authority at all. Planting this now makes the timer feel necessary rather than arbitrary.

2.3 Dual-Mode Operation

Protection begins with a single hardware bit. The bit records which of two worlds the CPU is currently executing in, and the instruction set itself changes depending on its value.

Definition (Dual-mode operation). *A hardware* ***mode bit*** *records the current privilege level. The kernel runs in* ***kernel mode*** *(mode bit = 0); user programs run in* ***user mode*** *(mode bit = 1). Some instructions are legal only when the bit is* 0*.*

Concepts That Click

The two modes differ in what they may execute and touch. Kernel mode carries the full instruction set, access to all memory and all devices, and the ability to change the mode bit. User mode carries a restricted instruction set, access only to the process's own address space, and no direct way to change the bit. The restriction is not a convention that well-behaved programs respect; it is enforced by the hardware on every instruction. The mode bit is *hardware state*, not an ordinary memory variable (Silberschatz et al., Ch. 1 §1.5, pp. 20–23).

Who may execute what?

The instruction set is partitioned accordingly. Table 2.1 lists the two halves.

Privileged (kernel only)	**Unprivileged (user or kernel)**
• set or clear the mode bit	• arithmetic and logic
• disable or enable interrupts	• loads and stores to the process's own memory
• load the timer	• branches and function calls
• issue raw I/O instructions	• read the clock (not set it)
• change the page-table base	• execute a trap instruction
• halt the machine	

Table 2.1: The privilege partition. A privileged instruction executed in user mode traps to the kernel rather than succeeding silently.

Two entries deserve comment. Loading the timer is privileged: user control would make preemption optional, since the program could set the next expiry beyond its own lifetime. Changing the page-table base is privileged: it would let a process redirect translation to another process's frames. The last unprivileged entry is the important one — a user program may execute a trap instruction, the only legal door into the kernel (Section 2.5). A privileged instruction executed in user mode does not fail silently; it traps to the kernel, which normally terminates the process.

> **Teaching note:** Write the mode-bit polarity on the board and keep it there: kernel = 0, user = 1. Students who have met the opposite convention elsewhere will invert it under exam pressure. What matters is not the numeric value but that the bit is hardware state outside the user's writable address space; say that explicitly, and tell the class the exam uses this course's convention.

The mode transition graph

Figure 2.1 shows the complete set of legal transitions between the two modes.

Build order (from the coordinate map): place the user box at $(-3.4, 0)$ and the kernel box at $(3.4, 0)$, each labelled with its mode-bit value; draw the mid-height return edge from user east to kernel west; then the upper arc from user north to kernel north; then the lower arc from user south to kernel south. Narration: "Start where the program lives: user mode, bit one. The only way up is an event — a trap or system call on the upper arc, an interrupt or exception on the lower arc. The only way back is the return-from-trap, which restores the saved context and the user bit." Point out what is *not* drawn: no user-controlled edge into the kernel, and no edge that bypasses the hardware.

Entry into the kernel is always a **controlled** hardware event. A trap or system call is deliberate; an interrupt or exception is forced. In every case the hardware, not user code, performs the mode switch and selects the entry point. The return path restores the saved user context and the user mode bit. The kernel does not "call" user code; it returns to it.

Concepts That Click

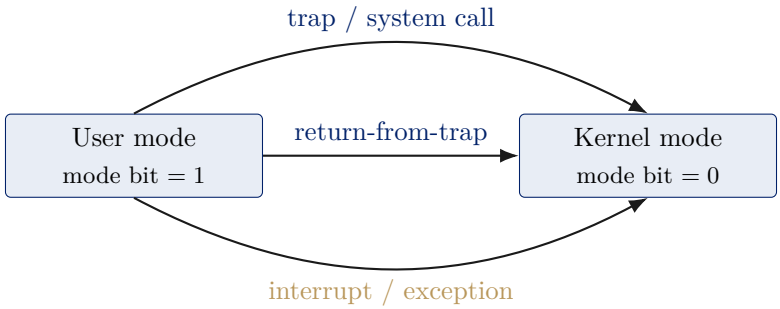


Figure 2.1: The mode transition graph. Entry into the kernel is always a controlled hardware event; the return path restores the saved user context and the user mode bit.

Board: Run the privilege classification drill. On the board, in order: (1) draw two columns, "kernel only" and "any mode"; (2) read the eight items one at a time and let the class vote before you write; (3) place each under the winning column. Items: (a) add two registers; (b) disable interrupts; (c) read the clock; (d) load the timer; (e) execute `syscall`; (f) change the page-table base; (g) branch to a function; (h) halt. Answers: (a) any, (b) kernel, (c) any, (d) kernel, (e) any, (f) kernel, (g) any, (h) kernel. Narration: "Safe operations touch only your own state; dangerous ones touch shared state or the mechanisms of control." Items (c) and (e) generate the best argument.
Board: Trace one `write` as a two-lane swimlane. On the board, in order: (1) draw a vertical boundary and label the lanes "user" and "kernel"; (2) write `write(1, buf, n)` in the user lane; (3) draw the argument registers, then the trap arrow crossing the boundary; (4) in the kernel lane write "hardware saves PC/flags", "validate", "perform I/O", "store return value"; (5) draw the return-from-trap arrow; (6) write "library sets errno" in the user lane. Narration: "The crossing is a hardware event, not a function call: user code does not choose the entry address and cannot return to an arbitrary instruction inside the kernel." Ask which step could not be predicted from the C code (the hardware context save). This rehearses Lecture 3.

> **Teaching note:** This is the moment for Socratic Check 1. Pose it exactly as written: "Why can't the mode bit live in ordinary memory? Describe the attack that becomes possible if it does." Collect two or three answers before revealing the sequence: the program stores `0` into the variable, disables interrupts, reprograms the timer, overwrites the page tables, and the hardware no longer has a trusted state to return to. The bit must live outside the address space user code can write. If a student says "the OS could check the variable," push back: the check would run on the same compromised machine.

The consequence is worth its own sentence, because students file protection under "policy." Protection is not a policy the OS can choose to enforce; it is a property the hardware guarantees. The OS decides *what* to protect, but the mode bit decides *whether* it can. This small, hardware-enforced boundary is the foundation of the **trusted computing base** — the set of components the system must trust (Stallings, Ch. 3 §3.5, p. 140).

> **Teaching note:** Keep the three event words straight, because the deck's graph deliberately groups them and the exam will separate them. A **trap** is software-initiated and deliberate (the `syscall` instruction); an **interrupt** is asynchronous and external (the timer); an **exception** is synchronous and unintended (a privileged instruction attempted in user mode, or a bad address). All three enter the kernel through hardware; only the trap is requested by the program. Expect students to call all three "interrupts"; correct it once, precisely, and move on.

Concepts That Click

2.4 The Timer and Preemption

The loop that never leaves user mode

The runaway program is a different kind of problem from the ones dual-mode operation solves. `while(1);` executes only unprivileged instructions. It never traps, never calls the OS, and never touches a device. Every instruction it executes is legal in user mode. If nothing external interrupts it, the kernel never runs again, and the machine is not corrupted — it is simply monopolized. The OS needs an event that fires *without* the user program's cooperation. That event is the **timer interrupt**.

Definition (Timer interrupt). *An asynchronous event raised by a hardware timer when its count reaches zero. It switches the CPU into kernel mode and transfers control to the kernel's timer handler, regardless of what user code is doing.*

The mechanism has five steps. A hardware timer counts down independently of the instruction stream. When it reaches zero, it raises a **timer interrupt**. The interrupt switches the CPU to kernel mode and jumps to the kernel's timer handler. The handler saves the running process's context and calls the scheduler. Loading the timer is a **privileged** instruction, so user code cannot set it. The timer is a device, not a software alarm. If user code could reload it, preemption would be optional — and a malicious program would opt out (Silberschatz et al., Ch. 1 §1.5, pp. 20–23; Stallings, Ch. 3 §3.5, p. 140).

A quantum on the timeline

Figure 2.2 places one scheduling interval, called a quantum, on a timeline.

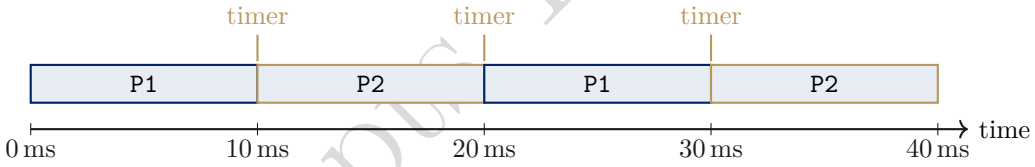


Figure 2.2: Two processes sharing one CPU in 10 ms quanta. The timer fires at 10, 20, and 30 ms, even when the running process is still runnable.

Definition (Quantum). *The fixed interval of CPU time that the scheduler grants one process before the timer interrupt forces a re-evaluation. In this lecture each process receives a* 10 *ms quantum.*

Build order (from the coordinate map): draw the time axis and its five ticks first, so the scale is visible before any block exists; then shade the P1 block (0–10 ms), then P2, P1, P2; finally add the three gold timer marks. Narration: "P1 runs, the timer fires at ten milliseconds whether P1 is finished or not, so P2 gets the CPU; at twenty it fires again." Every block ends at a gold tick, not at a natural completion — that is what "forced" means. The process sees continuous execution; the kernel sees a sequence of bounded intervals.

Each process receives one 10 ms quantum. The timer fires at the end of every quantum, even if the process is still runnable. The scheduler decides who runs next; the timer only guarantees that the decision happens. In Figure 2.2, `P1` and `P2` alternate every 10 ms, with timer interrupts at 10, 20, and 30 ms.

Definition (Preemption). *The kernel forcibly takes the CPU away from a running process at a timer interrupt, saves its context, and may schedule a different process.*

Concepts That Click

Cooperative scheduling makes yielding voluntary: a process yields when it chooses to, and one infinite loop stops the system. Preemptive scheduling makes yielding unnecessary: the timer fires, and the kernel keeps control of the CPU whether or not the process wanted to give it up. The difference is not politeness; it is who holds the power to decide. The choice of quantum is a trade-off. A large quantum wastes response time: an interactive process waits longer before it next runs. A tiny quantum wastes CPU time on context switches. The next example makes the second cost quantitative.

Example 2.4.1 (The Overhead of a Context Switch)**.** *A context switch costs* 12 μs*. The quantum is* Q*. What fraction of CPU time is spent switching?*

Let Q *be measured in microseconds. The CPU completes* $1/Q$ *quanta per microsecond, hence* $10^6/Q$ *quanta per second, and one context switch occurs at each quantum boundary. The switching time per second is therefore*

$$\left(\frac{10^6}{Q}\right)(12\ \mu s) = \frac{12 \times 10^6}{Q}\ \mu s.$$

As a fraction of one second (10^6 μs*), the overhead is* $12/Q$*. Equivalently,*

$$overhead = \frac{12\ \mu s}{Q},$$

provided Q *is expressed in microseconds.*

Quantum Q	***Switches per second***	***Overhead***
100 μs	10,000	120,000 $\mu s = 12\%$
1 *ms*	1,000	12,000 $\mu s = 1.2\%$
10 *ms*	100	1,200 $\mu s = 0.12\%$

For $Q = 100$ μs*, there are* 10,000 *switches per second; at* 12 μs *each that is* 120,000 $\mu s = 12\%$ *of the second. For* $Q = 1$ *ms the same calculation gives* 1.2*, and for* $Q = 10$ *ms,* 0.12%*. The fixed cost is divided by a larger* Q *as the quantum grows — which is why the quantum cannot simply be made as small as possible.*

Board: Build a second quantum timeline with different numbers: a 5 ms quantum, three processes, 0–30 ms. On the board, in order: (1) draw the axis with ticks every 5 ms; (2) mark the timer ticks; (3) fill the blocks round-robin — P1 0–5, P2 5–10, P3 10–15, P1 15–20, P2 20–25, P3 25–30; (4) circle every instant at which the kernel has control: 5, 10, 15, 20, 25, 30 ms. Narration: "The pattern is longer but the rule is identical: every block ends at a timer tick, and every tick is a moment when the kernel could change its mind." Erase one block and ask who runs next. This is a constructed variant of the deck's two-process figure.

Board: Shade the switching fraction on a one-second bar. On the board, in order: (1) draw a rectangle labelled "one second of CPU time"; (2) for $Q = 250$ μs, compute $10^6/250 = 4{,}000$ switches $\times\ 12$ μs $= 48{,}000$ μs $= 4.8\%$, and shade that sliver; (3) for $Q = 2$ ms, compute $500 \times 12 = 6{,}000$ μs $= 0.6\%$, and shade a visibly smaller sliver; (4) label both. Narration: "The small quantum buys responsiveness by spending almost five percent of the CPU on switching; the two-millisecond quantum spends well under one percent." Ask which an interactive login prompt prefers and which a batch job prefers; require the reason.

Concepts That Click

Teaching note: Pose Socratic Check 2 before revealing the answer: "If a small quantum improves response time, why not 1 μs?" Let the class commit to a number. Then compute $12/1 = 1200\%$: the CPU spends twelve times as long switching as executing. Add the costs the formula omits — cache and TLB state are destroyed on every switch, and the scheduler itself consumes CPU — and conclude that real systems choose 1–100 ms.

2.5 Protection in Depth

The mode bit and the timer protect the CPU. A complete boundary also protects memory and the devices.



Figure 2.3: The three resources the kernel protects and the hardware mechanism behind each.

Build order (from the coordinate map): place the three resource boxes left to right at $(-4.2, 0)$, $(0, 0)$, and $(4.2, 0)$; add the three gold mechanism labels beneath them at $y = -1.15$; then point from each box to its label while narrating: "CPU time is guarded by the timer; memory by the MMU; I/O by privileged instructions." The figure is spare because the point is the one-to-one correspondence. If asked where the mode bit belongs, answer that it is the precondition for all three: every mechanism below assumes the CPU can distinguish kernel from user code.

Memory protection

Three organizations are common, all enforcing the same rule at different granularities. With **contiguous allocation**, base and limit registers bound the process and the hardware checks every address. With **paging**, the page table maps only the process's legal pages, and an unmapped reference traps. With **segmentation**, each segment has its own base and limit, and the hardware checks the offset. The pattern is the same: user code never sees a physical address; every access is translated and checked, and a violation becomes a **trap**, not a silent write. Details belong to Unit III (Lectures 20–25).

Board: Check three addresses against a base/limit pair. On the board, in order: (1) write "base = `0x10000`", "limit = 64 KB = `0x10000`"; (2) draw a bar from `0x10000` to `0x20000`; (3) mark the test addresses `0x10000`, `0x1FFFF`, `0x20000` below it; (4) evaluate: the base byte and the last byte are legal, the one-past-the-end address is not. Narration: "The limit register holds the *size* here, not the last legal address — make that convention explicit before you grade." The illegal access traps to the kernel, which normally terminates the process; a silent write would corrupt whatever lay beyond the limit. Trapping converts invisible corruption into an attributable fault.

I/O protection

All raw I/O instructions are privileged. A user program cannot talk to a disk, network card, or timer directly. The user-level `read` and `write` calls are system calls: the kernel validates the buffer, device, and count, then invokes the device driver, which programs the controller. `cat` never touches the controller; it asks the kernel, which decides whether the request is legal. Direct access would let

Concepts That Click

a process bypass file permissions, corrupt another process's data, or hang the device for everyone; privileging the instruction makes validation unavoidable.

Protection versus security

Protection is a *mechanism*; security is a *policy* that uses it (Table 2.2). Protection is internal — memory isolation, privilege levels, access control — enforced by the OS and hardware. Security faces external threats — authentication, malware defense, firewalls — and is enforced by policy, cryptography, and administrative practice. A perfect mode bit does not help if the administrator grants every user root access (Stallings, Ch. 3 §3.6, p. 143).

Protection	**Security**
Internal mechanisms among processes	External threats and attackers
memory isolation, privilege levels, access control	authentication, malware defense, network firewalls
Enforced by the OS and hardware	Enforced by policy, cryptography, and administrative practice

Table 2.2: Protection is the mechanism; security is the policy that uses it (Stallings, Ch. 3 §3.6, p. 143).

The system-call gate

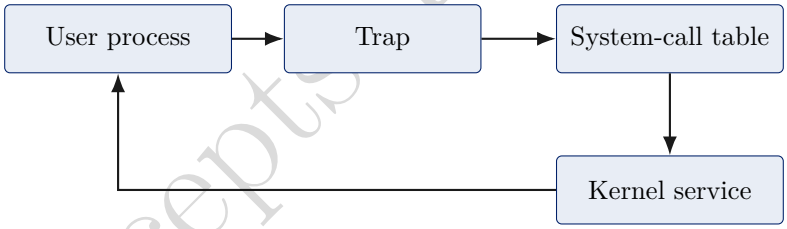


Figure 2.4: The system-call gate: a trap dispatches through the system-call table to a kernel service and returns through the same controlled entry point.

Build order (from the coordinate map): place the user process at $(-4.3, 0)$, the trap at $(-0.8, 0)$, the table at $(3.0, 0)$, and the service at $(3.0, -2.0)$; draw the forward path left to right then down; draw the return path westward and up into the user process. Narration: "Every request takes the same four hops — user, trap, table, service — and returns along the same path. The table makes the gate safe: the program supplies a *number*, not an address, so it cannot jump into the middle of the kernel." The trap is the **only legal door**; the kernel validates every argument and pointer; the return path restores user mode. Lecture 3 opens this door.

Board: Draw the legal path for `cat` as a three-box flow with a return arrow. On the board, in order: (1) write `cat` in a box; (2) write `read` beside it, with an arrow crossing a boundary labelled "trap"; (3) write "kernel validates fd/buffer/count"; (4) write "driver programs controller" below; (5) draw the return arrow to `cat`. Narration: "`cat` executes no privileged instruction. Privilege lives in the kernel; the program only asks, and the kernel checks descriptor, buffer, and count before the driver touches the controller." Erase the middle box and have a student restore the three validations.

Concepts That Click

Teaching note: Pose Socratic Check 3 here, before the board flow: "If all I/O instructions are privileged, how does `cat` read a file? Trace the legal path in three steps." A satisfactory answer names the library `read` and the trap, then the kernel's validation and dispatch to the driver, then the driver programming the controller and the return to user mode. A weak answer says "the kernel does it for `cat`" without naming the trap; press for the boundary crossing, because that is the examinable step.

2.6 Putting It Together: The Control Loop

Protection is not one mechanism. It is a loop: the timer fires, the kernel runs, the scheduler chooses, and user code resumes — or traps when it asks for something it cannot do.

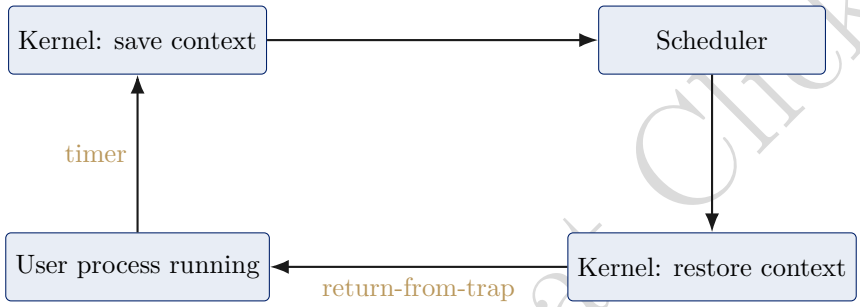


Figure 2.5: The full control loop: a timer interrupt transfers control to the kernel, the scheduler chooses, and a return-from-trap resumes user code.

Build order (from the coordinate map): place the four boxes at the corners of a rectangle; draw the left edge upward, labelling it "timer"; the top edge to the scheduler; the right edge down to "kernel: restore context"; the bottom edge back to the user, labelled "return-from-trap." Narrate it as a clock: "The loop runs on every timer tick and every blocking system call. The kernel is event-driven; the process cannot see the loop, only time passing." Both edge labels are hardware events; the scheduler is the only box where a *decision* is made.

Example 2.6.1 (Stopping the Runaway Process)**.** *`P1` enters `while(1);` in user mode at $t = 0$. The timeline unfolds as follows.*

1. *$t = 0$: `P1` enters `while(1);` in user mode.*
2. *$t = 10$ ms: the timer fires; hardware switches to kernel mode.*
3. *The kernel saves `P1`'s PC and registers in its process control block (PCB).*
4. *The scheduler picks `P2`; the kernel restores `P2`'s context and returns to user mode.*
5. *$t = 20$ ms: the timer fires again; the scheduler may pick `P1` or a third process.*
6. *If `P1` must be stopped, the kernel can deliver a signal or terminate it — it already has control.*

At no step is `P1` asked to cooperate. The timer is external, the save is kernel work, and the scheduler's choice is made in kernel mode.

Board: Trace round-robin control with three processes. On the board, in order: (1) draw a 0–40 ms axis with 10 ms ticks; (2) put `P1` on 0–10, `P2` on 10–20, `P3` on 20–30, `P1` again on 30–40;

Concepts That Click

(3) mark 10, 20, 30, 40 and write "kernel has control" above each; (4) ask the class to name every instant at which the kernel could terminate `P1`. Narration: "`P1` never yields, yet the kernel holds the CPU at four instants — stopping it is a kernel decision, not a favour." This is a constructed three-process trace of the deck's runaway mechanism.

Board: Blank-recall the loop, then insert the system-call edge. On the board, in order: (1) redraw Figure 2.5 with the four boxes and the two edge labels erased; (2) have a student reconstruct "timer" and "return-from-trap"; (3) add an entry edge labelled "trap / system call" and a second exit edge labelled "return-from-trap"; (4) ask which box the system-call path enters (the same kernel entry, via the trap handler). Narration: "The loop has two doors: the timer pushes the process out, and a system call lets it ask to come in. Either way the kernel saves context, decides, and returns." Do not fill in the reconstruction.

> **Teaching note:** Ask before drawing the three-process trace: "`P1` never yields. How many times can the kernel act on it in the first forty milliseconds?" Students often answer "never" or "once." Have them commit, then draw. The four instants are the payoff and directly answer the opening paradox. If time is short, this is the one board example not to skip.

2.7 Exit Ticket and Comprehension Checks

Use the deck's three Socratic Checks in order; they double as the exit ticket. (1) *Why can't the mode bit live in ordinary memory?* Expect the store-`0` attack, the consequences (interrupts disabled, timer reprogrammed, page tables overwritten), and the loss of any trusted state. (2) *Why not a* 1 μs *quantum?* Expect 12/1 = 1200%, cache/TLB destruction, and scheduler cost; real systems pick 1–100 ms. (3) *If I/O is privileged, how does* `cat` *read?* Expect `read` → trap → validation → driver → controller → return, with no privileged instruction from `cat`. Two more recur: "protection and security are the same thing" (see §2.5; Stallings, Ch. 3 §3.6, p. 143) and "a system call is a function call" (the trap is a hardware event with a fixed entry point — why Lecture 3 needs a full hour).

2.8 Summary: Fast Pre-Class Refresh

- **The design problem.** The kernel shares the CPU with the code it supervises, so it runs only on an event; protection must let untrusted code run at full speed while guaranteeing the kernel can regain control.
- **Dual-mode operation.** A hardware mode bit (kernel = 0, user = 1). Mode-bit change, interrupt control, timer load, raw I/O, page-table base, and halt are kernel-only; in user mode they trap. The bit must be hardware, or user code could write `0` and become the kernel (Silberschatz et al., Ch. 1 §1.5, pp. 20–23; Stallings, Ch. 3 §3.5, p. 140).
- **The timer and preemption.** User code can loop forever without a privileged instruction, so the kernel needs an external event: the timer interrupts, the handler saves context and calls the scheduler. Loop: timer (or trap) → save → schedule → restore → return.
- **The quantum trade-off.** Overhead is 12 $\mu s/Q$ — 12% at 100 μs, 1.2% at 1 ms, 0.12% at 10 ms; at 1 μs, 1200%, plus cache/TLB destruction.
- **Memory, I/O, and policy.** Base/limit, page tables, or segments check every access; raw I/O is privileged, so devices are reachable only through validated calls and drivers. Protection is the mechanism; security is the policy that uses it (Stallings, Ch. 3 §3.6, p. 143).

Concepts That Click